CS339: Abstractions and Paradigms for Programming

The Procedural Paradigm

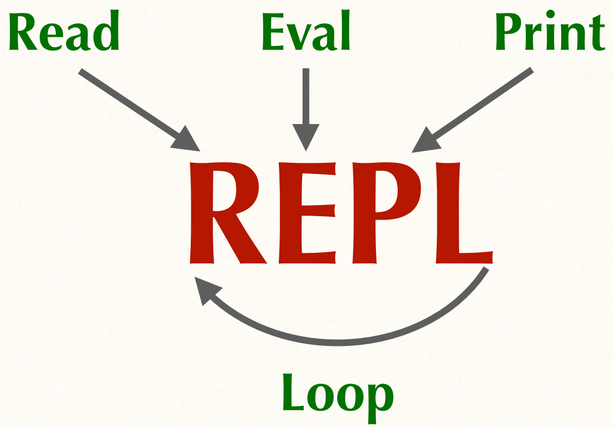
Manas Thakur
CSE, IIT Bombay

Autumn 2026

Recall and Observe

- DrRacket evaluates a *combination* in three steps:
  - Evaluate the operator
  - Evaluate the operands
  - Apply the procedure obtained by Step 1 to the operands obtained by Step 2
- Except for a few *special forms* (such as `define`).

- How do interpreters work?
- How do they differ from compilers?



Applying Procedures

To apply a compound procedure to arguments, evaluate the body of the procedure with each formal parameter replaced by the corresponding argument.

```
> (define (add x y)
    (+ x y))
> (add 4 7)
```

```
(+ 4 7)

11
```

Called the Substitution Model of Evaluation

Substitution Model: Practice

To apply a compound procedure to arguments, evaluate the body of the procedure with each formal parameter replaced by the corresponding argument.

```
> (define (foo x y)
    (* x (+ (* x 2) y)))
> (foo 4 5)
```

```
(* 4 (+ (* 4 2) 5))
(* 4 (+ 8 5))
(* 4 13)
52
```

A “Procedural” Example

```
> (define (square x) (* x x))

> (define (sum-of-squares x y)
    (+ (square x) (square y)))

> (define (f a)
    (sum-of-squares (+ a 1) (* a 2)))
```

Solve using substitution

```
(define (sum-of-squares x y)
  (+ (square x) (square y)))

(define (square x) (* x x))

(define (f a)
  (sum-of-squares (+ a 1) (* a 2)))
```

```
(f 5)
(sum-of-squares (+ 5 1) (* 5 2))
(+ (square (+ 5 1)) (square (* 5 2)))
(+ (* (+ 5 1) (+ 5 1)) (* (* 5 2) (* 5 2)))
(+ (* 6 6) (* 10 10))
(+ 36 100)
136
```

Could we have substituted this way?

```
(define (sum-of-squares x y)
  (+ (square x) (square y)))

(define (square x) (* x x))

(define (f a)
  (sum-of-squares (+ a 1) (* a 2)))
```

Applicative Order of Evaluation

```
(f 5)
(sum-of-squares (+ 5 1) (* 5 2))
(sum-of-squares 6 10)
(+ (square 6) (square 10))
(+ (* 6 6) (* 10 10))
(+ 36 100)
136
```

Evaluate arguments then apply

Solve using substitution

Normal Order of Evaluation

```
(define (sum-of-squares x y)
  (+ (square x) (square y)))

(define (square x) (* x x))

(define (f a)
  (sum-of-squares (+ a 1) (* a 2)))
```

```
(f 5)
(sum-of-squares (+ 5 1) (* 5 2))
(+ (square (+ 5 1)) (square (* 5 2)))
(+ (* (+ 5 1) (+ 5 1)) (* (* 5 2) (* 5 2)))
(+ (* 6 6) (* 10 10))
(+ 36 100)
136
```

Fully substitute then reduce

Applicative vs Normal Orders of Evaluation

Call by value

Call by name (need)

- Applicative order
  - avoids redundant computation
  - takes lesser memory

```
(f 5)
(sum-of-squares (+ 5 1) (* 5 2))
(sum-of-squares 6 10)
(+ (square 6) (square 10))
(+ (* 6 6) (* 10 10))
(+ 36 100)
136
```

```
(f 5)
(sum-of-squares (+ 5 1) (* 5 2))
(+ (square (+ 5 1)) (square (* 5 2)))
(+ (* (+ 5 1) (+ 5 1)) (* (* 5 2) (* 5 2)))
(+ (* 6 6) (* 10 10))
(+ 36 100)
136
```

- However, there are advantages of normal order too, and there are ways to make it more efficient (post mid-sem).

- Which one does Scheme use?
- How can you find out for any language?

Conditionals

- Absolute value of x:

$$|x| = \begin{cases} x, & \text{if } x > 0 \\ 0, & \text{if } x = 0 \\ -x, & \text{if } x < 0 \end{cases}$$

```
(define (abs x)
  (cond ((> x 0) x)
        ((= x 0) 0)
        ((< x 0) (- x))))
```

Clause / Predicate / Action

$$|x| = \begin{cases} -x, & \text{if } x < 0 \\ x, & \text{otherwise} \end{cases}$$

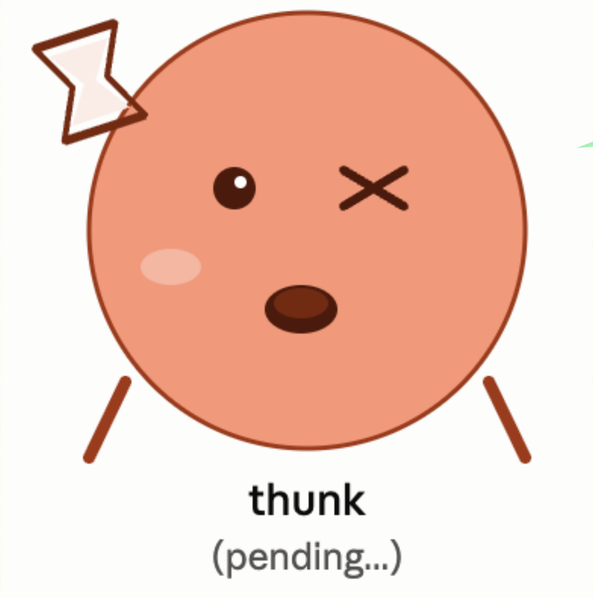
```
(define (abs x)
  (cond ((< x 0) (- x))
        (else x)))
```

OR

```
(define (abs x)
  (if (< x 0)
      (- x)
      x))
```

Predicate / Consequent / Alternate

Unevaluated…



Every loop you've ever written is something Scheme insists doesn't need to exist.